OLA: Observatorio Litoral de Anomalías Térmicas

Informe de avance de un sistema web de monitoreo y alerta de la temperatura superficial del mar desarrollado bajo un proceso de aseguramiento de la calidad

Frederick Mares Graos, Jhordan Huamani Huamani, Jorge Ortiz Castañeda,
Miguel Angel Flores Leon y Piero Adrian Delgado Chipana
Escuela Profesional de Ingeniería de Software, Facultad de Ingenierías y Arquitectura
Universidad La Salle, Arequipa, Perú
Curso de Calidad de Software (3.8.2.21) — Docente: Maribel Molina Barriga

***Resumen*—El Instituto del Mar del Perú (IMARPE) publica como dato abierto la anomalía diaria de la temperatura superficial del mar (ATSM) de sus laboratorios costeros, pero en un formato que solo aprovechan los especialistas. Este informe presenta el avance de OLA, una aplicación web que clasifica el estado térmico de los 10 laboratorios costeros del litoral peruano en cálido, neutro o frío, lo muestra en un mapa interactivo, grafica y compara su histórico desde 1970, estima la tendencia de los próximos días con dos modelos simples y avisa por correo a los usuarios suscritos cuando una zona mantiene una anomalía sostenida. El proyecto se desarrolló con Scrum y con un proceso de calidad basado en IEEE 830, ISO/IEC 25010 y una estrategia de pruebas por capas. Al cierre de este avance, los ocho requisitos funcionales están implementados y verificados por 943 pruebas automatizadas, con una cobertura del 95 % en el backend y sin violaciones serias de accesibilidad WCAG 2.2 AA. Con el dataset completo (125,701 registros) la importación tarda 11.8 s y el mapa se muestra en 1.07 s con una red 4G, frente a límites de 120 s y 3 s. El análisis previo de los datos reales obligó a precisar cinco de los ocho requisitos, y las pruebas detectaron defectos que habrían pasado inadvertidos, entre ellos una verificación de tipos que no revisaba ningún archivo.**

***Palabras clave*—anomalía de la temperatura superficial del mar, calidad de software, ingeniería de requisitos, ISO/IEC 25010, pruebas automatizadas, sistemas de alerta temprana, accesibilidad web.**

***Abstract*—The Peruvian Sea Institute (IMARPE) publishes the daily sea surface temperature anomaly (SSTA) of its coastal laboratories as open data, in a format only specialists can use. This progress report presents OLA, a web application that classifies the thermal state of Peru's 10 coastal laboratories as warm, neutral or cold, displays it on an interactive map, charts and compares their history since 1970, estimates the short-term trend with two simple models, and emails subscribed users when a zone shows a sustained anomaly. The project followed Scrum and a quality process based on IEEE 830, ISO/IEC 25010 and a layered testing strategy. All eight functional requirements are implemented and verified by 943 automated tests, with 95 % backend coverage and no serious WCAG 2.2 AA violations.**

***Palabras clave*—sea surface temperature anomaly, software quality, requirements engineering, automated testing, early warning systems.**

I. Introducción

El litoral peruano, bajo la influencia del sistema de la corriente de Humboldt, es muy sensible a las variaciones de la temperatura del mar. Un calentamiento o enfriamiento sostenido altera la disponibilidad de los recursos pesqueros y, con ella, la economía de miles de pescadores artesanales. El Niño Costero de 2017 causó pérdidas estimadas en 3,100 millones de dólares y afectó a 1.1 millones de personas; parte del problema fue que los sistemas de monitoreo no alertaron a tiempo sobre un fenómeno que se desarrolló frente a la costa, fuera de las regiones oceánicas que se vigilan habitualmente [1].

IMARPE mide a diario la temperatura del mar en sus laboratorios costeros y publica la anomalía en la Plataforma Nacional de Datos Abiertos [2]. El dato es público, pero llega como un archivo de más de cien mil filas, sin clasificar y con retrasos y huecos que un lector no especialista no puede advertir. No existe tampoco un mecanismo que avise a quien sigue una zona concreta cuando la temperatura se aparta de lo normal de forma sostenida.

OLA (Observatorio Litoral de Anomalías térmicas) responde a esa brecha con una aplicación web móvil primero que traduce el dato a un lenguaje sencillo. El proyecto es además el trabajo práctico del curso de Calidad de Software, por lo que su objetivo secundario es demostrar un proceso de calidad aplicado a lo largo de todo el desarrollo. OLA es una herramienta complementaria: no reemplaza los boletines de IMARPE ni pronostica eventos El Niño, función que corresponde al ENFEN [3].

Las contribuciones de este avance son: (i) un catálogo de requisitos trazable, refinado a partir del análisis del dataset real; (ii) un sistema funcional que implementa los ocho requisitos funcionales comprometidos; y (iii) evidencia cuantitativa de calidad (pruebas, cobertura, rendimiento y accesibilidad) vinculada a cada requisito.

El resto del informe se organiza así: la Sección II resume el marco teórico; la III, la metodología; la IV, los requisitos; la V, la arquitectura y los algoritmos; la VI, el avance del producto; la VII, la evaluación de calidad; y las Secciones VIII y IX, la discusión y las conclusiones.

II. Marco teórico y trabajos relacionados

A. Anomalía térmica y El Niño Costero

La ATSM es la diferencia entre la temperatura medida y su valor climatológico para la misma fecha y lugar. En el Perú, el ENFEN define los eventos El Niño y La Niña costeros a partir del Índice Costero El Niño (ICEN), una media móvil de tres meses de la anomalía en la región Niño 1+2 [3], [4]. OLA no calcula el ICEN: trabaja con la anomalía diaria de

cada laboratorio y adopta un umbral de $\pm 0.5\,^{\circ}\mathrm{C}$ para separar el estado neutro de los estados cálido y frío, como criterio simple y configurable.

B. Detección de eventos sostenidos

La literatura sobre olas de calor marinas define un evento como un periodo en que la temperatura supera un umbral durante al menos cinco días, y admite huecos cortos dentro del mismo evento [5]. OLA sigue la misma idea: un episodio de alerta exige una racha mínima de registros fuera del rango neutro (cinco por defecto) y tolera hasta dos días faltantes entre mediciones, porque el dataset de IMARPE tiene huecos frecuentes.

C. Información para comunidades pesqueras

El acceso oportuno a información a través del teléfono móvil tiene efectos medibles en la pesca artesanal: en Kerala, la adopción del celular redujo la dispersión de precios y el desperdicio de captura [6]. Esto respalda la decisión de diseñar OLA para el celular y de comunicar los estados con palabras de la costa en lugar de términos técnicos.

D. Calidad de software

La especificación de requisitos sigue IEEE 830 [7] y las buenas prácticas de ISO/IEC/IEEE 29148 [8], con fichas de requisitos al estilo de Durán y Bernárdez [9]. Los atributos no funcionales se organizan según el modelo de calidad de producto de ISO/IEC 25010 [10]. Las pruebas siguen la pirámide de pruebas [11], [12]: muchas pruebas unitarias rápidas, menos de integración y pocas de extremo a extremo. La accesibilidad se mide contra WCAG 2.2 [13], y la distinción de series para personas con daltonismo se valida con simulación de deficiencias de la visión del color [14].

III. Metodología

A. Organización del trabajo

El equipo de cinco integrantes trabajó con Scrum [15], con un Product Owner, un Scrum Master que también actúa como responsable de QA y tres desarrolladores. El semestre se divide en siete sprints de dos semanas y dos hitos de control: el Hito 1 (catálogo de requisitos y avance del producto), al que corresponde este informe, y el Hito 2 (artículo final, prototipo y presentación).

B. Ingeniería de requisitos

El catálogo inicial definió ocho requisitos funcionales (RF-01 a RF-08) con un límite de ocho impuesto por el enunciado del curso. Antes de escribir código se analizó el archivo real de IMARPE. Ese análisis reveló cinco hechos que la especificación daba por sentados de otra manera (Tabla I) y motivó seis versiones controladas del catálogo, cada una registrada con fecha, requisito afectado y motivo.

Tabla I
Hallazgos del análisis del dataset y su efecto en los requisitos

Hallazgo	Consecuencia
125,701 filas de 1970 a 2026, no las ∼3,650 previstas	Carga por lotes y agrupado automático de series (RF-05, RF-08)
Archivo UTF-8 con marca de orden de bytes	El importador lee `utf-8-sig`
Matarani dejó de medir en 2016	Estado «sin datos recientes» derivado de la fecha (RF-04)
Huecos de hasta 1,127 días	Racha contada por registros; la línea del gráfico se corta (RF-01, RF-05)
El diccionario nombra un laboratorio inexistente en los datos	El catálogo de zonas se fija por migración y se rechazan laboratorios desconocidos (RF-08)

Tabla II
Requisitos funcionales y estado al Hito 1

ID	Requisito	Compl.	Estado
RF-01	Detección de anomalía térmica sostenida	Alta	Verificado
RF-02	Proyección de la tendencia a 3–7 días	Alta	Verificado
RF-03	Notificación por correo y en la aplicación	Alta	Verificado
RF-04	Mapa interactivo de las 10 zonas	Media	Verificado
RF-05	Gráficos históricos por laboratorio	Media	Verificado
RF-06	Comparación de 2 a 4 laboratorios	Media	Verificado
RF-07	Registro, autenticación y suscripción	Baja	Verificado
RF-08	Importación del CSV de ATSM	Baja	Verificado

C. Estrategia de verificación

Cada requisito se verifica en la capa donde su lógica vive. La lógica de dominio es pura: recibe listas y fechas explícitas, sin acceso a la base de datos ni al reloj, lo que hace sus pruebas rápidas y deterministas. Las pruebas de integración ejercitan los endpoints contra una base PostgreSQL separada. Como las bibliotecas de mapa y gráficos no pueden dibujarse en un DOM simulado, su comportamiento real se comprueba con pruebas de extremo a extremo en Chromium, en escritorio y en un celular emulado, que incluyen análisis de accesibilidad con axe y regresión visual. El correo se verifica con entrega real a un servidor de pruebas (Mailpit), no con un simulacro.

IV. Requisitos del sistema

La Tabla II resume los requisitos funcionales con su complejidad y su estado. El catálogo completo, con objetivos, actores, requisitos de información, 18 reglas de negocio, 14 casos de uso, 19 requisitos no funcionales y matrices de trazabilidad, se entrega como documento aparte.

Los requisitos no funcionales principales son: carga del mapa en 3 s o menos con 4G (rendimiento), importación de un año de histórico en menos de dos minutos, cumplimiento de WCAG 2.2 AA con ningún estado comunicado solo por color (usabilidad), fecha del último dato siempre visible (fiabilidad), contraseñas con *hash* bcrypt, tráfico HTTPS y rechazo a arrancar con secretos débiles (seguridad), y despliegue íntegro en contenedores (portabilidad).

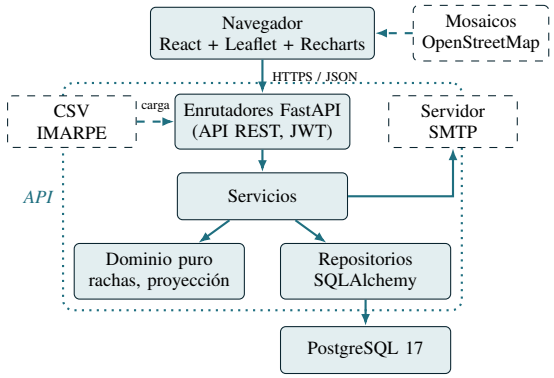


Figura 1. Arquitectura de OLA. El paquete de dominio no depende de la base de datos ni del reloj.

V. Arquitectura y diseño

A. Arquitectura

OLA sigue una arquitectura cliente-servidor en capas (Fig. 1). El frontend es una aplicación de una sola página en React, TypeScript y Vite, con Leaflet para el mapa y Recharts para los gráficos. El backend es una API REST en Python 3.13 con FastAPI, organizada en enrutadores, servicios, repositorios y un paquete de dominio puro. La persistencia usa PostgreSQL 17 con migraciones de Alembic. Todo se ejecuta en contenedores Docker y se despliega en un VPS Linux administrado con Dokploy, con Traefik como proxy HTTPS.

B. Clasificación y fecha de referencia

Sea a la anomalía de una medición y u el umbral ($0.5\,^{\circ}\mathrm{C}$ por defecto). La clasificación es

$$c(a) = \begin{cases} \text{cálido} & \text{si } a > u, \\ \text{frío} & \text{si } a < -u, \\ \text{neutro} & \text{en otro caso,} \end{cases} \tag{1}$$

con límite exclusivo: una anomalía de exactamente $\pm u$ es neutra. Todos los cálculos usan como fecha de referencia d_{ref} la de la medición más reciente del sistema, no el reloj del servidor. IMARPE publica con retraso; con el reloj real, las diez zonas habrían aparecido «sin datos recientes» y el mapa entero en gris. Como efecto secundario, las pruebas son deterministas sin congelar el reloj.

El color de cada zona z en el mapa sale del promedio de sus mediciones en una ventana de w días (5 por defecto):

$$\bar{a}_z = \frac{1}{|M_z|} \sum_{m \in M_z} a_m, \tag{2}$$

donde M_z son las mediciones de z con $d_{\mathrm{ref}} - w < d_m \le d_{\mathrm{ref}}$. El promedio evita que un solo día atípico haga parpadear el estado. Una zona sin mediciones en los últimos 7 días no se clasifica.

Algoritmo 1 Detección de episodios sostenidos

Entrada: serie ordenada (d_i, a_i), umbral u, racha mínima k, tolerancia g
Salida: lista de episodios

$R \leftarrow [\,]$; $E \leftarrow [\,]$
for cada (d_i, a_i) **do**
  $s \leftarrow c(a_i)$ ▷ Ec. (1)
  if $s =$ neutro **then**
    CERRAR(R, E); $R \leftarrow [\,]$
  else if $R \neq [\,]$ **y** ($s \neq$ clase(R) **o** $d_i - d_{\text{últ}} - 1 > g$) **then**
    CERRAR(R, E); $R \leftarrow [(d_i, a_i)]$
  else
    agregar (d_i, a_i) a R
  end if
end for
CERRAR(R, E)
procedimiento CERRAR(R, E): si $|R| \ge k$, agregar a E el episodio (inicio, fin, clase, $|R|$, pico)

C. Detección de rachas

El Algoritmo 1 recorre la serie de cada zona. Una racha crece mientras las mediciones conserven la misma clase no neutra y el número de días faltantes entre dos mediciones consecutivas no supere la tolerancia g. Un episodio se registra cuando la racha alcanza k registros. La inserción de episodios usa una resolución de conflictos sobre (laboratorio, estado, fecha de inicio), de modo que reevaluar no duplica episodios ni cambia sus identificadores, condición de la que depende que los avisos no se reenvíen.

D. Proyección a corto plazo

Sobre las últimas $N = 30$ mediciones de la zona se calculan dos modelos [16]. La regresión lineal por mínimos cuadrados ajusta $\hat{a}(t) = \beta_0 + \beta_1 t$ usando la fecha real t, de modo que los huecos pesen en la pendiente, y extrapola h días (3 a 7, con 5 por defecto). La media móvil ponderada proyecta el nivel reciente:

$$\hat{a}_{\mathrm{MMP}} = \frac{\sum_{i=1}^{N} i\, a_i}{\sum_{i=1}^{N} i}, \tag{3}$$

donde $i = N$ es la medición más reciente (peso 30) e $i = 1$ la más antigua (peso 1). La regresión sigue la *dirección* de la serie y la media, su *nivel*; cuando discrepan, esa diferencia se muestra al usuario como medida de incertidumbre. La confianza es alta si la cobertura de la ventana es de al menos 70 %, media si es de al menos 40 % y baja en otro caso o si el último dato supera la vigencia. El tramo estimado se marca con tres señales redundantes: trazo punteado, fondo sombreado y aviso en texto.

E. Notificaciones

La evaluación de alertas solo *registra* los avisos pendientes (inicio y fin de cada episodio, uno por usuario y momento). El envío por SMTP es un paso separado, de forma que un servidor de correo lento o caído no bloquea la evaluación. Los envíos fallidos quedan marcados con su motivo y se reintentan en el siguiente intento.



Figura 2. Pantalla de inicio en escritorio con datos de prueba: resumen del estado, mapa de las 10 zonas y accesos a las zonas en alerta (RF-04).

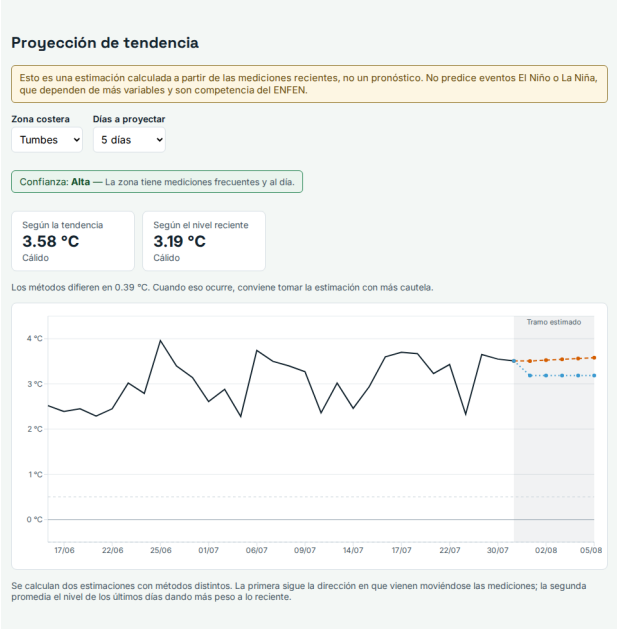


Figura 3. Proyección de tendencia (RF-02): estimación por regresión y por media ponderada, confianza y tramo estimado sombreado.

VI. Avance del producto

Al cierre de este avance, los ocho requisitos funcionales están implementados, integrados y desplegados. La Fig. 2 muestra la pantalla principal: el resumen indica la fecha del último dato de IMARPE y su antigüedad, las zonas en alerta y el número de zonas por estado; el mapa representa cada zona con color, símbolo y nombre, y resalta con un anillo las que tienen una alerta vigente. La Fig. 3 muestra la proyección con las dos estimaciones, su diferencia y el indicador de confianza.

Tabla III
Suites de pruebas automatizadas

Suite	Pruebas	Alcance
Backend, unitarias	212	Dominio puro, sin base de datos
Backend, integración	230	Endpoints, persistencia, permisos
Frontend (Vitest)	332	Componentes y páginas, con axe
Extremo a extremo	167	Playwright en escritorio y celular, axe, regresión visual
Rendimiento	2	Playwright, build de producción con 4G
Total	**943**	

La interfaz se rediseñó con un enfoque móvil primero: barra de navegación inferior en el celular, detalle de la zona en una hoja inferior modal, textos en lenguaje de la costa («sobre lo normal» en lugar de «anomalía positiva») y carga diferida de cada pantalla, que redujo la portada de 272 a 170 kB comprimidos. El sistema completo se despliega con un único archivo de Docker Compose; las migraciones se ejecutan al arrancar bajo un bloqueo de PostgreSQL para que varias réplicas no migren en paralelo.

Con el dataset real, la evaluación detecta 4,164 episodios desde 1970 en unos 3 s, de los cuales 8 seguían vigentes a la fecha del último dato.

VII. Evaluación de la calidad

A. Pruebas y cobertura

La Tabla III resume las suites. Las 943 pruebas se ejecutan en contenedores; el backend alcanza una cobertura del 95 % de las sentencias y pasa las herramientas ruff y mypy en modo estricto sin observaciones. El único módulo sin cobertura automática es el de arranque de producción, que se verificó levantando el *stack* desde una base vacía.

B. Rendimiento y accesibilidad

La Tabla IV compara las métricas medidas con los límites de la especificación. El tiempo del mapa se midió con la build de producción en un celular emulado con 4G (9 Mbps, 40 ms) y la CPU ralentizada al doble, como mediana de tres cargas en frío. El análisis de accesibilidad con axe es bloqueante en todas las pantallas; la única excepción documentada es el tamaño de los marcadores del mapa (WCAG 2.5.8), cubierta por la misma selección disponible en tarjetas y tabla. La paleta de las series comparadas mantiene, con cuatro series simultáneas, una diferencia de color $\Delta E \geq 9.0$ en deuteranopía y un contraste de al menos 3:1 contra el fondo.

C. Defectos detectados por el proceso

Las pruebas encontraron defectos que habrían llegado a producción. Los más significativos fueron: un decimal escrito con coma que se cargaba en silencio con un valor erróneo; una fecha repetida que abortaba toda la importación; peticiones tardías que sobrescribían el resultado vigente al cambiar de zona; sesiones rechazadas de forma intermitente porque

Tabla IV
REQUISITOS NO FUNCIONALES MEDIDOS

Métrica	**Límite**	**Medido**
Mapa con 10 zonas (4G)	$\leq 3\,\mathrm{s}$	$1.07\,\mathrm{s}$
Importación del dataset	$\leq 120\,\mathrm{s}$/año	$11.8\,\mathrm{s}$ (56 años)
Puntos por serie (56 años)	—	$< 1{,}000$
Portada comprimida	—	$170\,\mathrm{kB}$
Violaciones serias de axe	0	0
Cobertura del backend	—	95 %

el reloj de la máquina virtual retrocedía, resuelto con una tolerancia de 30 s; y dos procesos que chocaban al crear la cuenta de administrador. Dos defectos del propio proceso de pruebas resultaron los más instructivos: la verificación de tipos del frontend apuntaba a un archivo de configuración que no incluía archivos y pasaba sin revisar nada, y una tolerancia visual relativa del 0.5 % dejaba pasar cambios reales de texto.

VIII. DISCUSIÓN

El proyecto se implementó antes de cerrar el catálogo de requisitos, que se formalizó y amplió después a partir del código, las pruebas y el registro de decisiones. Esta ingeniería inversa de requisitos tiene un riesgo claro: documentar lo construido en lugar de lo necesario. Para mitigarlo, cada requisito se contrastó con la problemática original, cada cambio de alcance quedó registrado con su motivo y cada criterio de aceptación se vinculó a pruebas que fallan si el comportamiento cambia.

La lección más útil fue leer los datos reales antes de diseñar. Cinco de los ocho requisitos cambiaron por hechos del dataset, y la mayoría de esos cambios hacen el sistema más honesto: marcar zonas sin datos recientes, cortar la línea en los huecos o bajar la confianza de una proyección con datos antiguos evita que el sistema aparente una certeza que no tiene. La segunda lección es que una prueba verde solo vale si verifica lo que se cree: dos de los defectos encontrados hacían que las pruebas pasaran sin comprobar nada.

Entre las limitaciones actuales están: la capacidad de 50 usuarios concurrentes no se ha verificado con una prueba de carga; no hay integración continua y las pruebas se ejecutan en local; la importación es solo manual; los modelos de proyección son deliberadamente simples y no constituyen un pronóstico; y cuatro pantallas conservan el estilo anterior al rediseño.

IX. CONCLUSIONES Y TRABAJO FUTURO

OLA transforma un dataset público de difícil lectura en información útil para la pesca artesanal: el estado térmico de cada zona, su historia, una estimación de los próximos días y avisos cuando una anomalía se sostiene. Al Hito 1, los ocho requisitos funcionales están implementados y trazados a 943 pruebas automatizadas, y los requisitos de rendimiento y accesibilidad se cumplen con holgura.

Para el Hito 2 se prevé ejecutar una prueba de carga que verifique la capacidad concurrente, incorporar integración continua, extender el rediseño a las pantallas restantes y validar la comprensión de la interfaz con usuarios reales del perfil de pescador artesanal.

AGRADECIMIENTOS

Al Instituto del Mar del Perú y al Ministerio de la Producción, por publicar el dataset de ATSM como dato abierto, y a los colaboradores de OpenStreetMap, por el mapa base.

REFERENCIAS

[1] I. J. Ramírez and F. Briones, "Understanding the El Niño Costero of 2017: The definition problem and challenges of climate forecasting and disaster responses," *International Journal of Disaster Risk Science*, vol. 8, no. 4, pp. 489–492, 2017.

[2] Instituto del Mar del Perú (IMARPE) and Ministerio de la Producción (PRODUCE), "Anomalía de la temperatura superficial del mar (ATSM) de los laboratorios costeros del IMARPE," Plataforma Nacional de Datos Abiertos. [En línea]. Disponible: https://www.datosabiertos.gob.pe, 2026.

[3] Comisión Multisectorial ENFEN, "Definición operacional de los eventos El Niño costero y La Niña costera en el Perú," ENFEN, Lima, Perú, Nota Técnica 01-2024, 2024.

[4] K. Takahashi, K. Mosquera, and J. Reupo, "El índice Costero El Niño (ICEN): historia y actualización," *Boletín Técnico: Generación de modelos climáticos para el pronóstico de la ocurrencia del Fenómeno El Niño*, vol. 1, no. 2, pp. 8–9, 2014.

[5] A. J. Hobday, L. V. Alexander, S. E. Perkins, D. A. Smale, S. C. Straub, E. C. J. Oliver *et al.*, "A hierarchical approach to defining marine heatwaves," *Progress in Oceanography*, vol. 141, pp. 227–238, 2016.

[6] R. Jensen, "The digital provide: Information (technology), market performance, and welfare in the South Indian fisheries sector," *The Quarterly Journal of Economics*, vol. 122, no. 3, pp. 879–924, 2007.

[7] IEEE, "IEEE recommended practice for software requirements specifications," IEEE Std 830-1998, 1998.

[8] ISO/IEC/IEEE, "Systems and software engineering — life cycle processes — requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.

[9] A. Durán Toro and B. Bernárdez Jiménez, "Metodología para la elicitación de requisitos de sistemas software, versión 2.3," Universidad de Sevilla, Sevilla, España, Tech. Rep., 2002.

[10] ISO/IEC, "Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models," ISO/IEC 25010:2011, 2011.

[11] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Boston, MA, USA: Addison-Wesley, 2009.

[12] M. Fowler, "Test pyramid," [En línea]. Disponible: https://martinfowler.com/bliki/TestPyramid.html, 2012.

[13] World Wide Web Consortium (W3C), "Web content accessibility guidelines (WCAG) 2.2," W3C Recommendation. [En línea]. Disponible: https://www.w3.org/TR/WCAG22/, 2023.

[14] G. M. Machado, M. M. Oliveira, and L. A. F. Fernandes, "A physiologically-based model for simulation of color vision deficiency," *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 6, pp. 1291–1298, 2009.

[15] K. Schwaber and J. Sutherland, "The Scrum guide," [En línea]. Disponible: https://scrumguides.org, 2020.

[16] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.